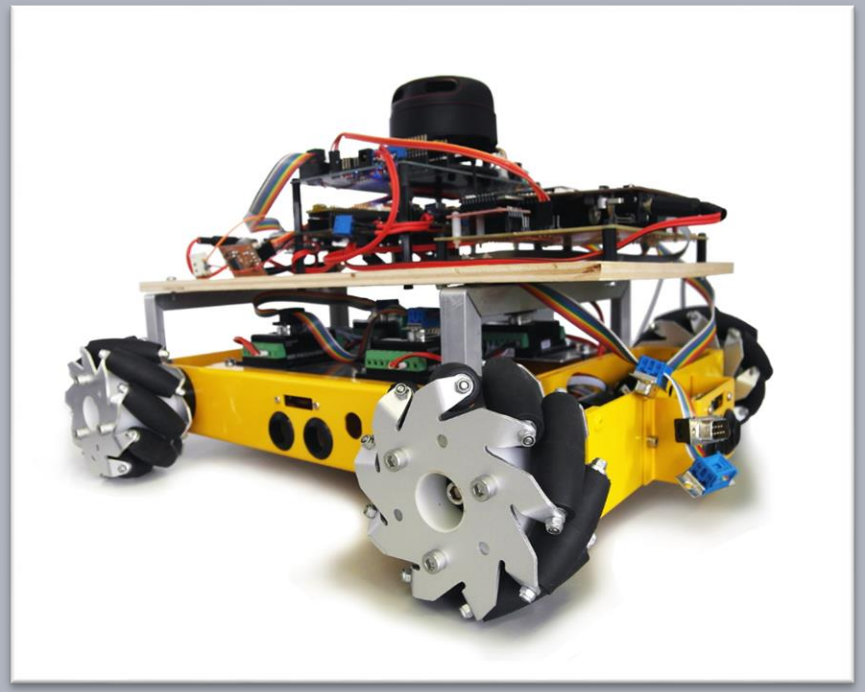
Advanced Embedded Architectures and Applications

Application Layer – Joystick Control

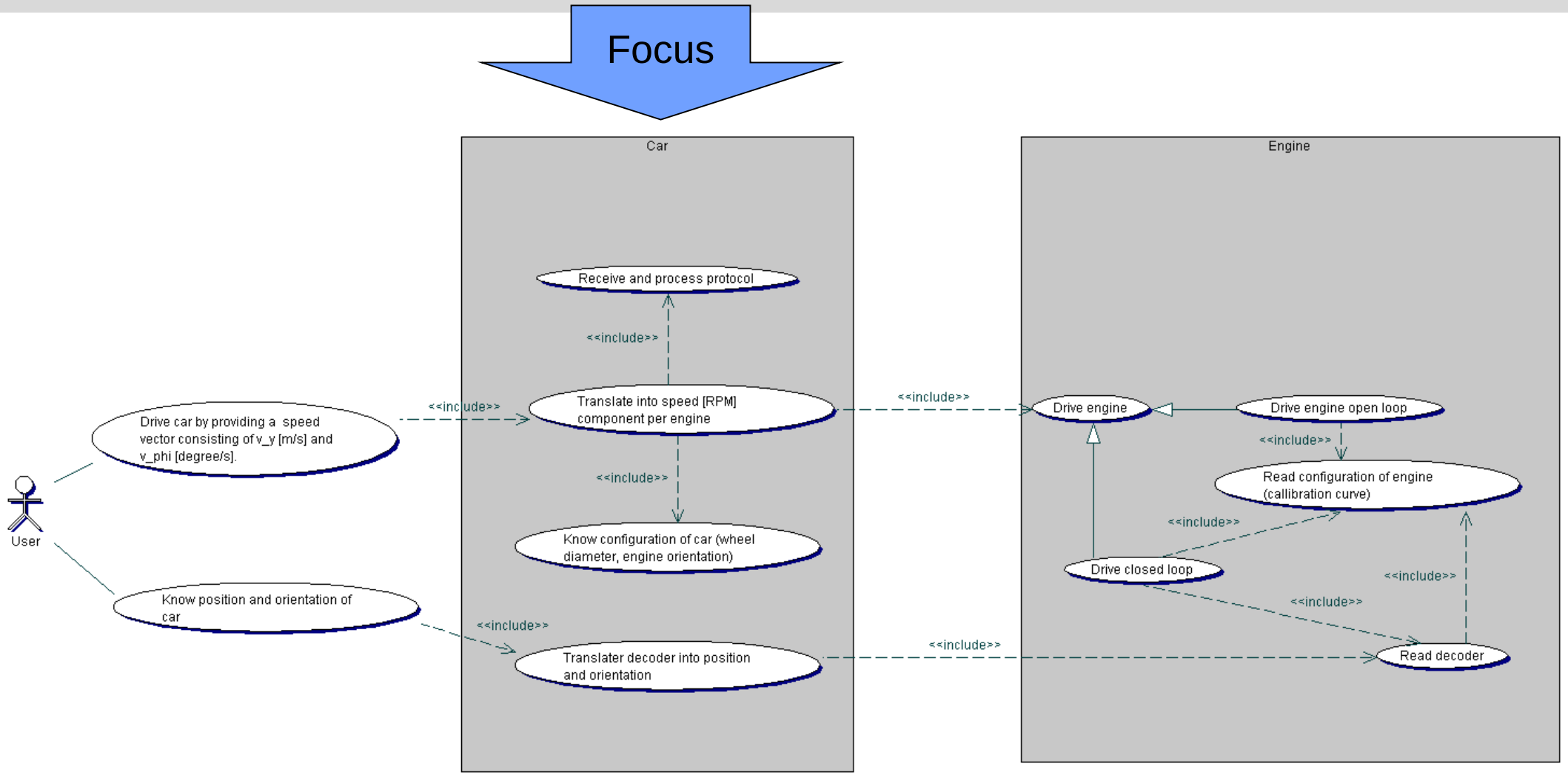
Prof. Dr.-Ing. Peter Fromm



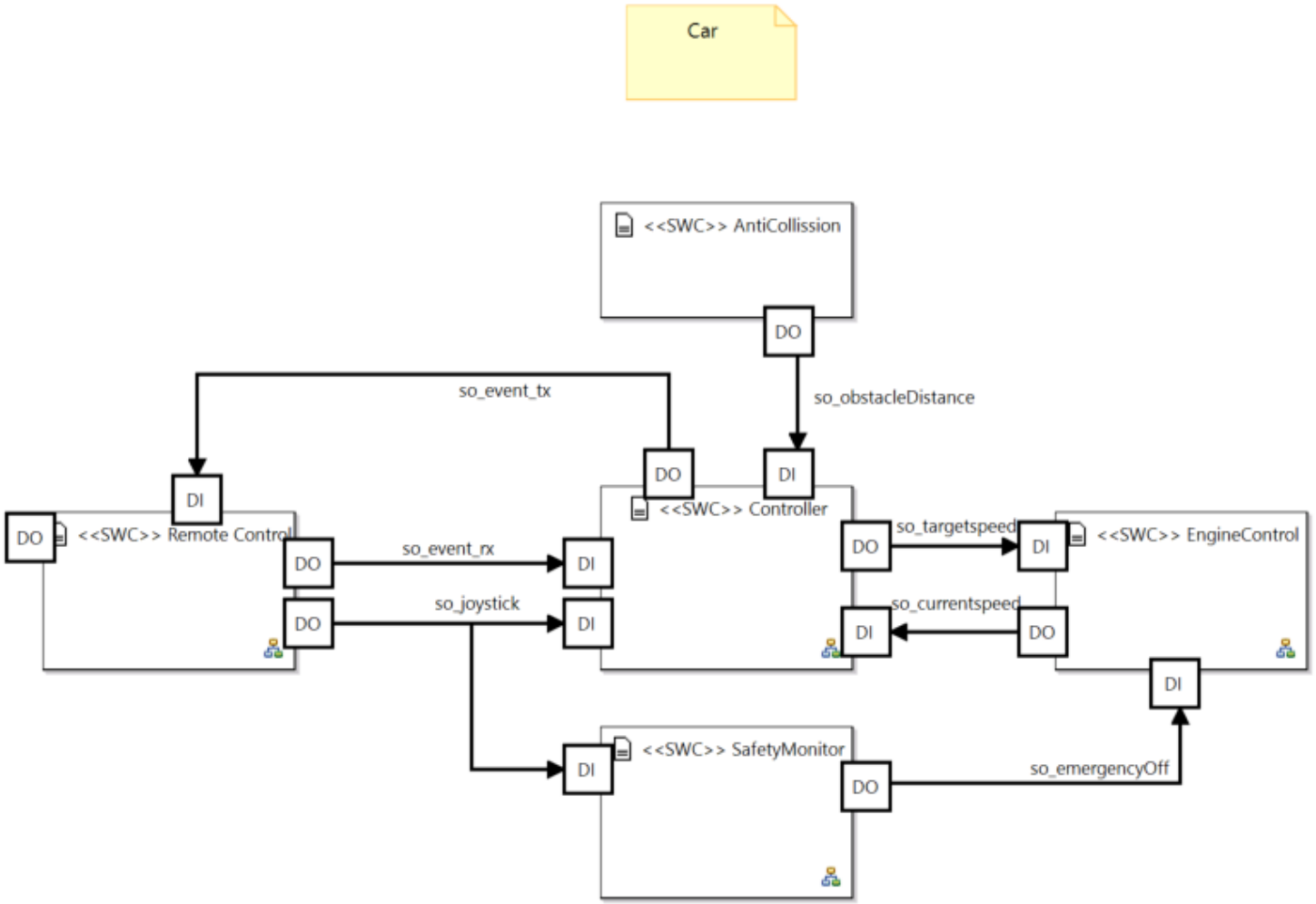
Content

- Getting the car operational
- Application development
- Implementing remote control
- And there it goes….
- Extension towards 1 Remote and n-Cars

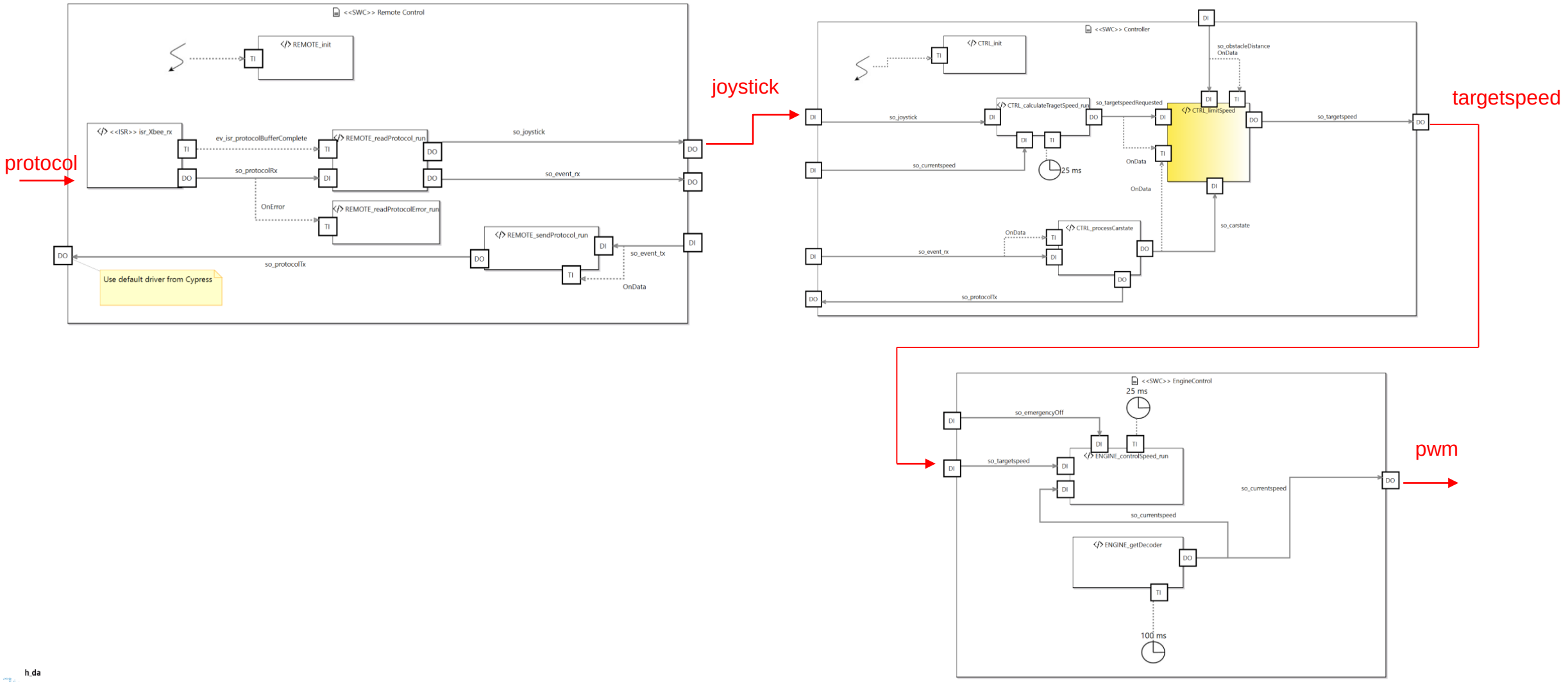
Where we want to go



What we have…


Car
<<SWC>> AntiCollission
DO
so_obstacleDistance
so_event_tx
DI
DO
DI
<<SWC>> Controller
DO
<<SWC>> Remote Control
so_event_rx
DO
DI
so_joystick
DO
DI
so_targetspeed
DO
DI
<<SWC>> EngineControl
so_currentspeed
DI
DO
DI
DI
<<SWC>> SafetyMonitor
so_emergencyOff
DO

The skinny sheep…

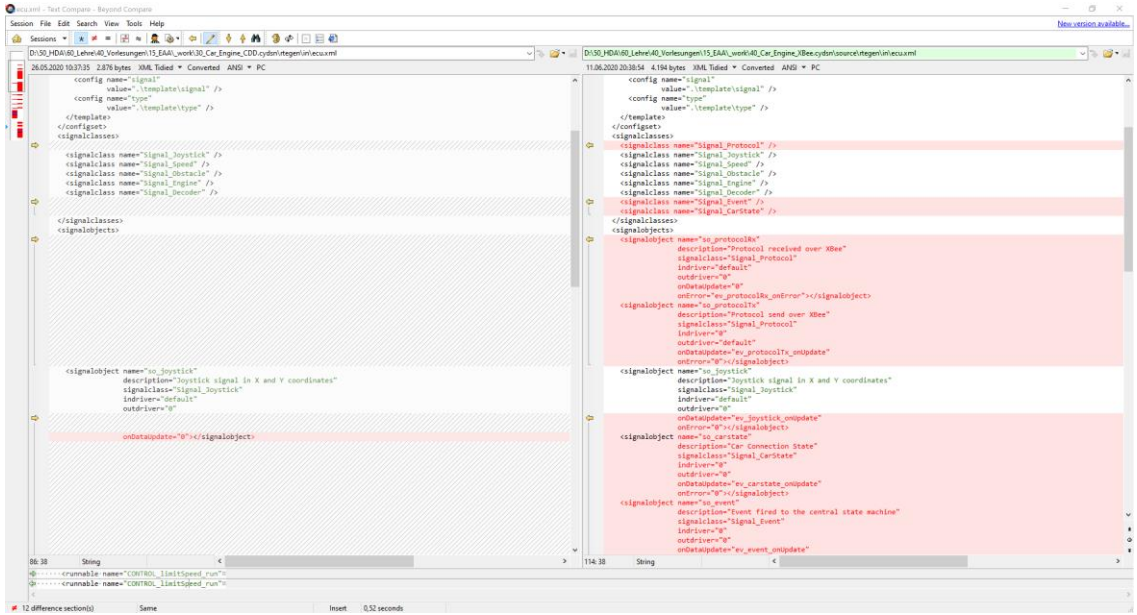


What do we have to change

New signal classes
- Signal_Protocol
- Signal_Joystick

New signal objects
- so_protocolRx
- so_protocolTx
- so_Joystick

Runnables
- Updated triggers
- REMOTE_read/send Protocol
- REMOTE_readProtocolError



Strategy

Step 1: Generate new RTE and get code compile-able (stubbing)

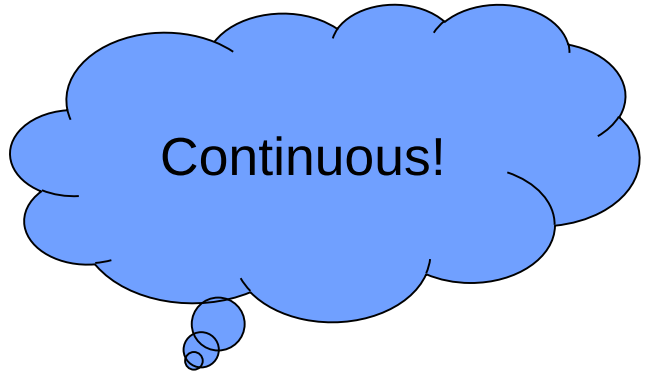
Step 2: Get communication up and running

Step 3: Create Framework for Remote application

Step 4: Skinny Sheep – transmit and process joystick signal only

Step 5: Scaling – decide on data units and scale data as needed

Step 6: Add state machine logic for connection
and multiple driving algorithms (next lecture)



Creating the Remote Application

As we have similar features, we will try to reuse as much code as possible.

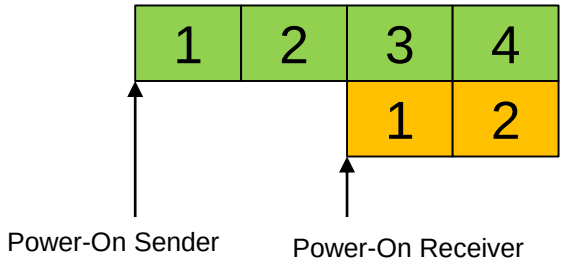
- Remote runnables and signals
- Joystick signal
- Event signals

Reusing the code will also allow us to maintain the system without code rot. If we change a common part for one system, we will simply copy it to the other project!

Only the signal flow needs to be redirected. Easy going using the RTE…

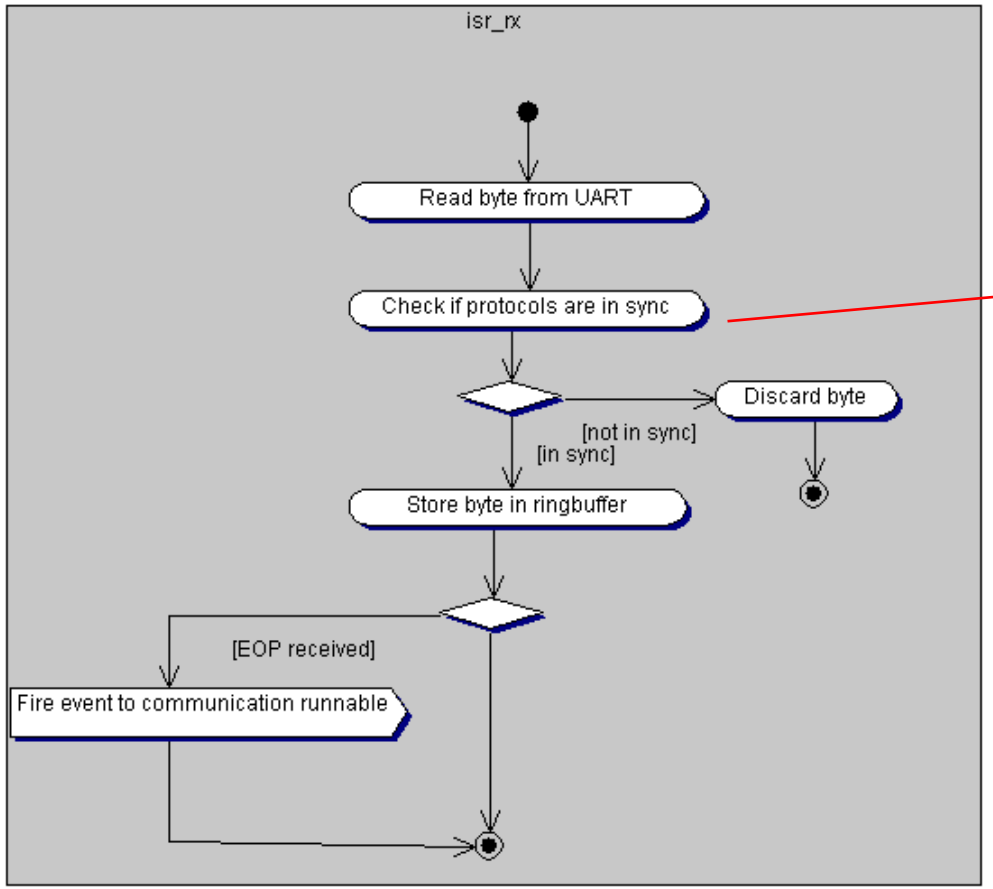
The challenge of device synchronisation

- As the remote control and the car are booted at different times, it might happen that the remote already starts sending without the car receiving.
- Furthermore, after loosing a byte during transmission, we also face the problem of lost synchronization.
- Sender
- Receiver



A Communication protocol it a state machine. The state represents the byte we expect at a given position. We have to keep the sender and receiver state machine in sync! And we have to be able to resync after communication errors.

Initial protocol synchronization



Discarding data might not always be the best option! Alternatives?

```
/** ------------------------ Protocol synchronisation --------------------------------- **/

static boolean_t REMOTE__isSynced = FALSE;

/**
 * As the sender and receiver possibly do not start at the same time and we might suffer from boot-bytes,
 * all bytes are ignored until we find the first true byte in the sequence
 * later on, this concept may also be used for resynchronisation
 * \param uint8_t data  : IN Byte currently received
 * \ return: TRUE if system is in sync, false otherwise
 */
boolean_t SIGNAL_PROTOCOL__SyncProtocol(uint8_t data)
{
    //Identify valid start of protocol identifier
    if (PROT_ID_Connection == data ||
        PROT_ID_Joystick   == data)
    {
        REMOTE__isSynced = TRUE;
    }

    return REMOTE__isSynced;
}
```

Car: Parsing the Joystick Protocol

```
void REMOTE_readProtocol_run()
{
    //Read in the protocol data
    SIGNAL_PROTOCOL_data_t prot = SIGNAL_PROTOCOL_INIT_DATA;

    RC_t result = RTE_SIGNAL_PROTOCOL_pullPort(&SO_PROTOCOLRX_signal);

    if (RC_SUCCESS != result)
    {
        LOG_W("REMOTE_read","error %d", result);
    }

    prot = RTE_SIGNAL_PROTOCOL_get(&SO_PROTOCOLRX_signal);

    //LOG_I("Prot:","%c %c %c %d",prot.m_id, prot.m_sender, prot.m_fid, prot.m_length);

    //Now let's process the content
    if (prot.m_id == PROT_ID_Joystick)
    {
        SIGNAL_JOYSTICK_data_t joystick = SIGNAL_JOYSTICK_INIT_DATA;
        joystick.m_x = (sint8_t)prot.m_payload[0];
        joystick.m_y = (sint8_t)prot.m_payload[1];

         //LOG_I("Joystick:","%d %d ",joystick.m_x, joystick.m_y);

        RTE_SIGNAL_JOYSTICK_set(&SO_JOYSTICK_signal, joystick);
    }

    //Todo connection protocol
}
```

Note the debugging trace…

Error handling to be improved!

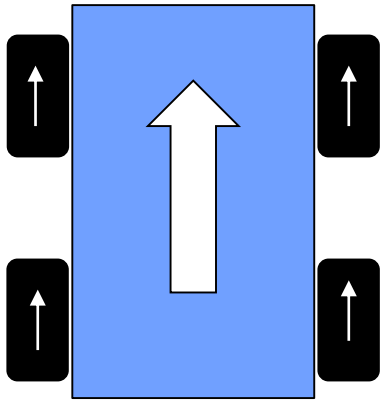
Same hardware, no issue with data representation. Cast ok.

Car: Calculating Target Speed

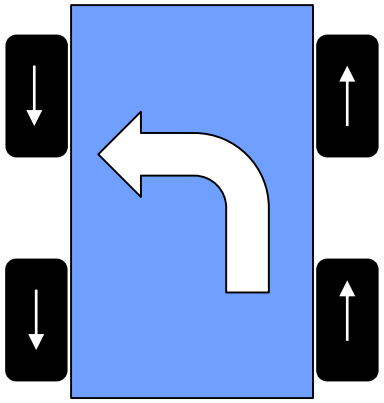
```
/* Runnables */
void CONTROL_calculateTargetspeed_run()
{
    //Read the joystick data from the RTE
    SIGNAL_JOYSTICK_data_t joystick = RTE_SIGNAL_JOYSTICK_get(&SO_JOYSTICK_signal);

    //Calculate the requested target speed
    SIGNAL_SPEED_data_t targetSpeed_requested = SIGNAL_SPEED_INIT_DATA;

    //The joystick y-value represents the forward speed
    //The joystick x-value represents the car rotation
    targetSpeed_requested.m_vy = CAR_MAXSPEED * (float)joystick.m_y / CAR_MAXJOYSTICK;
    targetSpeed_requested.m_vphi = CAR_MAXROTATION * (float)joystick.m_x / CAR_MAXJOYSTICK;

    //LOG_I("CONTROL_calculateTargetspeed_run","vy = %d vphi = %d", (sint32_t)(targetSpeed_requested.m_vy*1

    //Send the target speed requested signal to the limitSpeed runnable
    RTE_SIGNAL_SPEED_set(&SO_TARGETSPEEDREQUESTED_signal, targetSpeed_requested);
}


void CONTROL_limitSpeed_run()
{
    //LOG_I("CONTROL_limitSpeed_run","");

    //Read the requested signal from the RTE
    SIGNAL_SPEED_data_t targetSpeed = RTE_SIGNAL_SPEED_get(&SO_TARGETSPEEDREQUESTED_signal);

    //At the moment, we simply pipe the data through

    //Write the limited signal to the RTE
    RTE_SIGNAL_SPEED_set(&SO_TARGETSPEED_signal, targetSpeed);
}
```

Car: Setting the engine speed

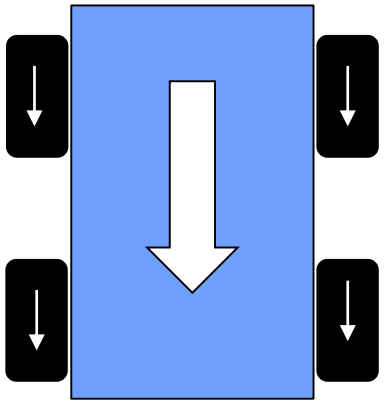
Forward



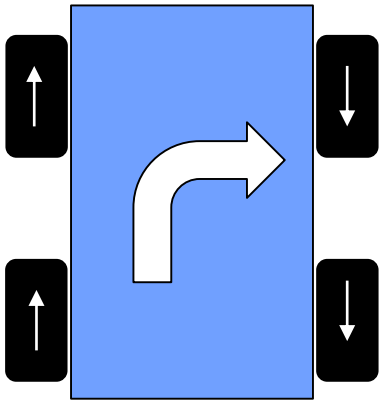
Clockwise



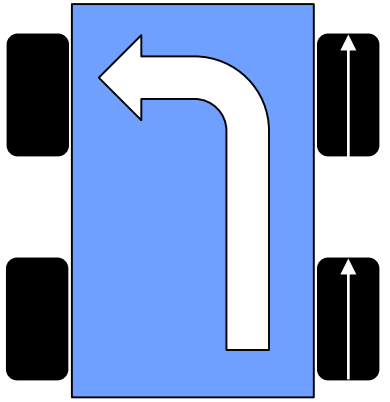
Reverse



Anti-Clockwise



Forward + Clockwise



Car: Setting the engine speed (first draft)

```
void ENGINE_controlSpeed_run()
{

    //Read in the targetspeed and check for validity
    SIGNAL_SPEED_data_t targetspeed = SIGNAL_SPEED_INIT_DATA;

    targetspeed = RTE_SIGNAL_SPEED_get(&SO_TARGETSPEED_signal);

    //LOG_I("ENGINE_setSpeed_run","ts %d %d",targetspeed.m_vy, targetspeed.m_vphi);

    //Translate into engine speed
    SIGNAL_ENGINE_data_t engine = SIGNAL_ENGINE_INIT_DATA;

    //Impact of longitude value vy
    engine.m_rpm_fl = K_VY2RPM * targetspeed.m_vy;
    engine.m_rpm_fr = K_VY2RPM * targetspeed.m_vy;
    engine.m_rpm_rl = K_VY2RPM * targetspeed.m_vy;
    engine.m_rpm_rr = K_VY2RPM * targetspeed.m_vy;


    //Impact of the rotation
    engine.m_rpm_fl += K_VPHI2RPM * targetspeed.m_vphi;
    engine.m_rpm_fr -= K_VPHI2RPM * targetspeed.m_vphi;
    engine.m_rpm_rl += K_VPHI2RPM * targetspeed.m_vphi;
    engine.m_rpm_rr -= K_VPHI2RPM * targetspeed.m_vphi;



    //Write to hardware
    RTE_SIGNAL_ENGINE_set(&SO_ENGINE_signal, engine);
    RTE_SIGNAL_ENGINE_pushPort(&SO_ENGINE_signal);
}
```

Adding some stability to the engine controller

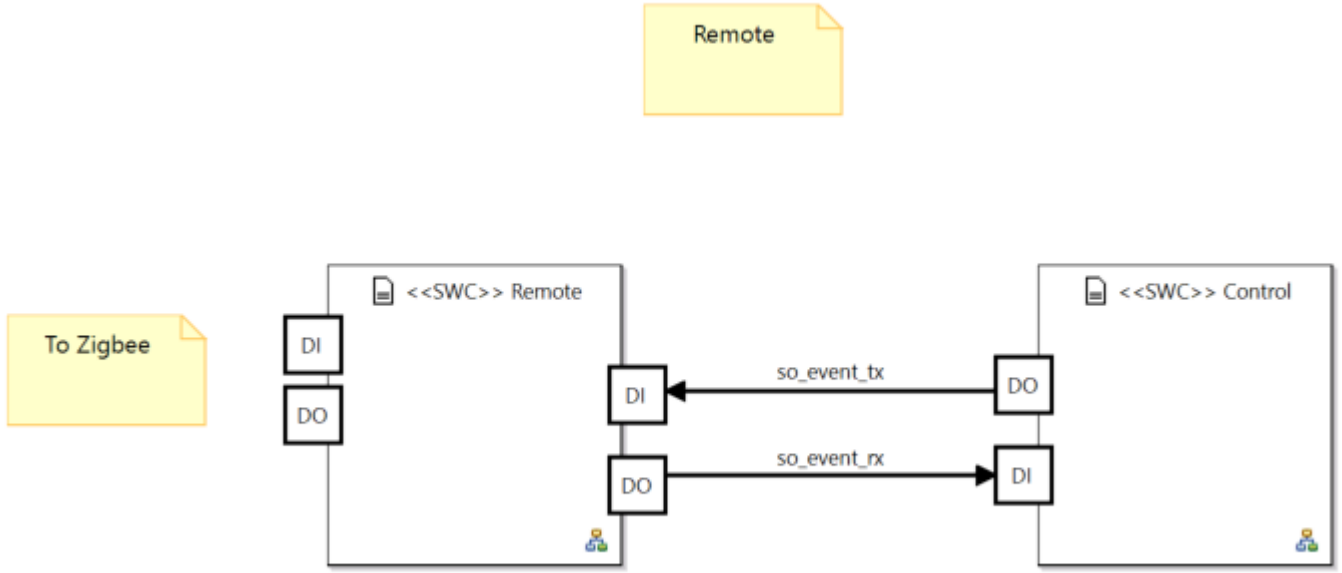
```
RC_t ENG_SetEngineClosedLoopPID(ENG_id_t id, uint16_t ticktime, sint16_t targetspeed, sint16_t* measuredspeed, sint16_t* controlspeed)
{

    float32_t error = 0;
    static float32_t integralError[4] = {0};
    static float32_t lastError[4] = {0};
    float32_t differentialError = 0;
    RC_t result;

    //Get the current speed
    result = ENG_GetSpeed(id, ticktime, measuredspeed);
    if (RC_SUCCESS != result)
    {
        return result;
    }

    //Limit low speed as controller tends to overreact
    if (abs(targetspeed) < ENG_MINRPM) targetspeed = 0;

    //Control logic
    error = targetspeed - (*measuredspeed);
    integralError[id] += ((error * ticktime) / 1000);
    differentialError = (float32_t)(error - lastError[id]) * (1000/ticktime);
    lastError[id] = error;

    //Limit I-part
    if (integralError[id] < -(ENG_PWM_MAXDUTY / ENG_CONTROL_KI)) integralError[id] =  -(ENG_PWM_MAXDUTY / ENG_CONTROL_KI);
    if (integralError[id] >  (ENG_PWM_MAXDUTY / ENG_CONTROL_KI)) integralError[id] =   (ENG_PWM_MAXDUTY / ENG_CONTROL_KI);

    //Caluclate new controlspeed
    *controlspeed = (sint16_t)(ENG_CONTROL_KP * error + ENG_CONTROL_KI * integralError[id] + ENG_CONTROL_KD * differentialError);

    //Avoid beebing
    if (abs(*controlspeed) < ENG_PWMZERO ||
       targetspeed == 0)
    {
        integralError[id] = 0;
        *controlspeed = 0;
    }

    result = ENG_SetPWM(id, *controlspeed);
    if (RC_SUCCESS != result)
    {
        return result;
    }

    return RC_SUCCESS;
}
```

Ignore too low target speeds, as this causes issues with the holding torque

The I-error may cause an overrun (float casted to sint16)

Too low control values cause beeping of the engine

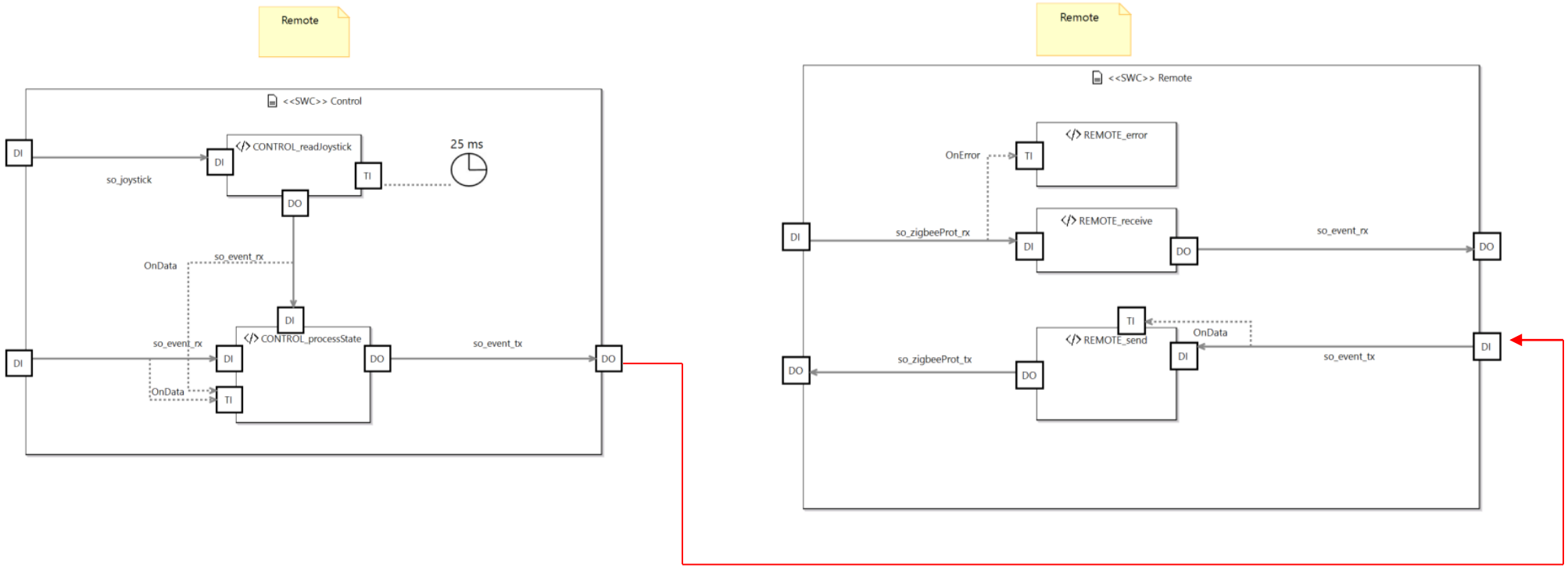
How about the remote control?



A bit more details…

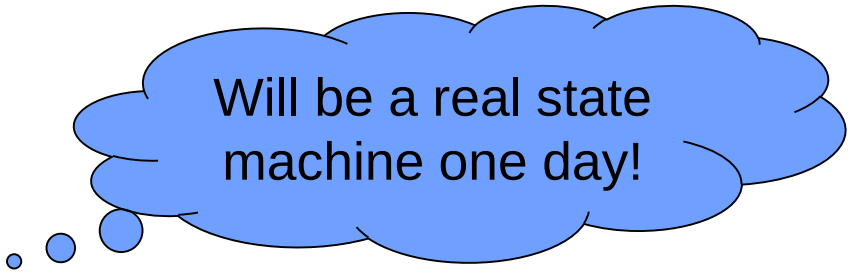
New…

Same like car…



Remote: Getting the joystick signal

```
/* Runnables */
void CONTROL_readJoystick_run()
{
    RC_t result = RC_ERROR;

    //Let's get the physical joystick value
    SIGNAL_JOYSTICK_data_t joystick = SIGNAL_JOYSTICK_INIT_DATA;
    result = RTE_SIGNAL_JOYSTICK_pullPort(&SO_JOYSTICK_signal);

    if (RC_SUCCESS != result)
    {
        //Todo error handling
        return;
    }

    //Everything ok, let's pack the joystick into a message (event signal) and send it to the state machine
    joystick = RTE_SIGNAL_JOYSTICK_get(&SO_JOYSTICK_signal);

    //Create the corresponding event
    SIGNAL_EVENT_data_t event = SIGNAL_EVENT_INIT_DATA;
    event.m_ev = EV_Joystick;
    event.m_lengthPayload = 2;
    event.m_payload[0] = (uint8_t)joystick.m_x;
    event.m_payload[1] = (uint8_t)joystick.m_y;

    //Note: as we send to the DI port of the state machine, we use the rx signal
    RTE_SIGNAL_EVENT_set(&SO_EVENT_RX_signal, event);
}


void CONTROL_processState_run()
{
    RC_t result = RC_ERROR;


    //First test, transmit joystick independant of state
    SIGNAL_EVENT_data_t event = SIGNAL_EVENT_INIT_DATA;
    event = RTE_SIGNAL_EVENT_get(&SO_EVENT_RX_signal);
    RTE_SIGNAL_EVENT_set(&SO_EVENT_TX_signal, event);
}
```



Remote: Transmitting the joystick event

```
void REMOTE_sendProtocol_run()
{
    //Read in internal message
    SIGNAL_EVENT_data_t event = SIGNAL_EVENT_INIT_DATA;
    event = RTE_SIGNAL_EVENT_get(&SO_EVENT_TX_signal);

    //Translate message into protocol
    SIGNAL_PROTOCOL_data_t prot = SIGNAL_PROTOCOL_INIT_DATA;

    if (EV_Joystick == event.m_ev)
    {
        prot.m_id = PROT_ID_Joystick;
        prot.m_sender = PROT_sender_remote;
        prot.m_fid = PROT_FID_Joystick;
        prot.m_length = event.m_lengthPayload;

        //Copy payload
        for (uint8_t i = 0; i < prot.m_length; i++)
        {
            prot.m_payload[i] = event.m_payload[i];
        }
    }

    RTE_SIGNAL_PROTOCOL_set(&SO_PROTOCOLTX_signal, prot);
    RTE_SIGNAL_PROTOCOL_pushPort(&SO_PROTOCOLTX_signal);
}
```

The Event – Message pattern….

- Looks like an overkill. We could also transmit the data directly to the UART instead?

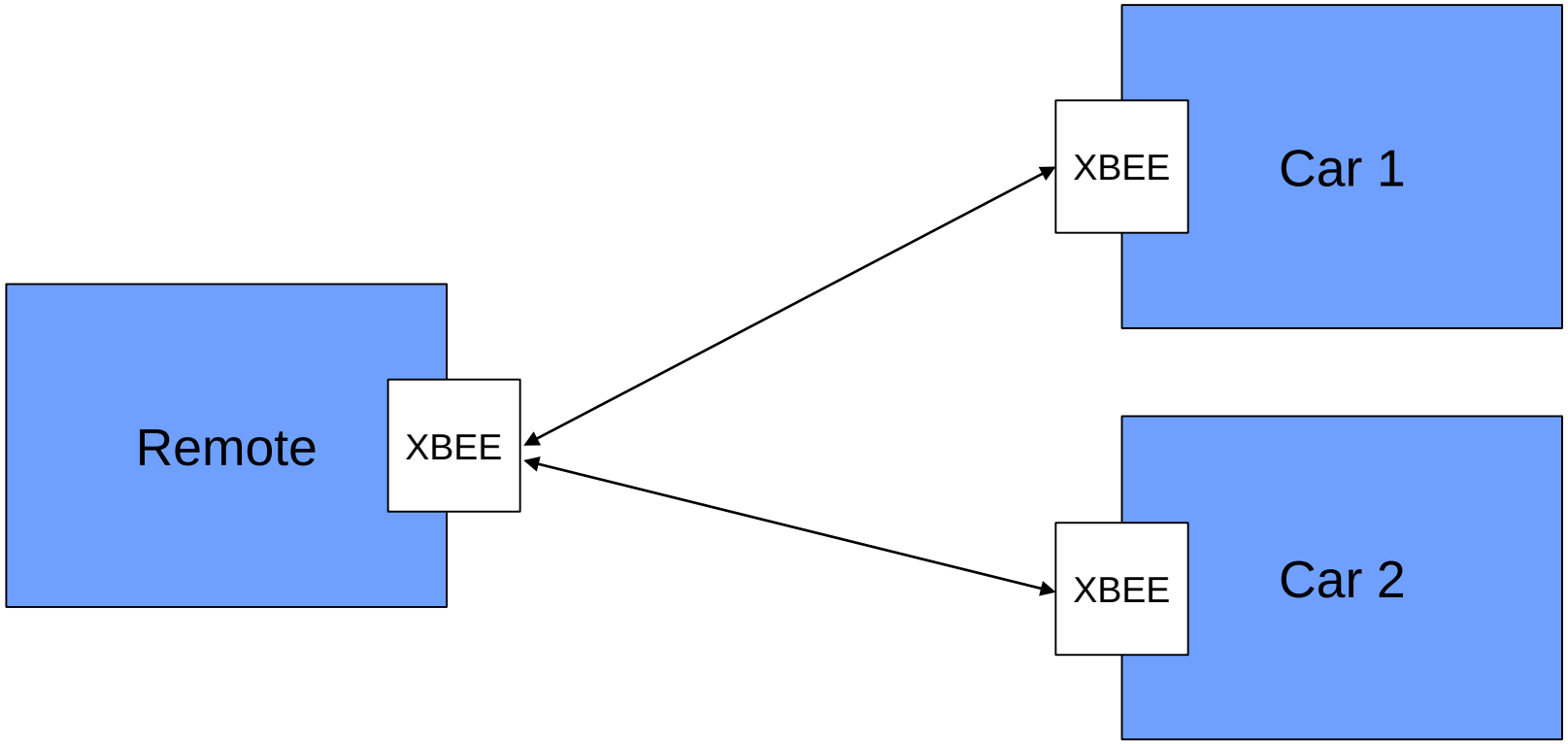
- Not really – consider the risk of races, i.e. one protocol preempts the sending of another protocol.
- Using the event-messages, we make sure, that one protocol is transmitted after the other, as the next event only will be processed, if the transmission runnable has finished working.

```
typedef enum {
    //Data send from Car to remote

    //Data send from Remote to Car
    EV_Joystick,
    EV_ConnectACK,
} EVENT_t;


#define EVENT_PAYLOADSIZE PROT_SIZEPAYLOAD

typedef struct
{
    EVENT_t m_ev;
    uint8_t m_lengthPayload;
    uint8_t m_payload[EVENT_PAYLOADSIZE];
} SIGNAL_EVENT_data_t;


#define SIGNAL_EVENT_INIT_DATA ((SIGNAL_EVENT_data_t){0})
```

It’s alive

Side-Notice: Now lets consider the following additional Use Case

- We have one remote and several cars
- The remote shall detect the cars
- And the select the one we want to drive



Xbee API Mode

In addition to the simple transparent mode we have used so far, Xbee supports a more complex API mode, which supports several services (some examples below)

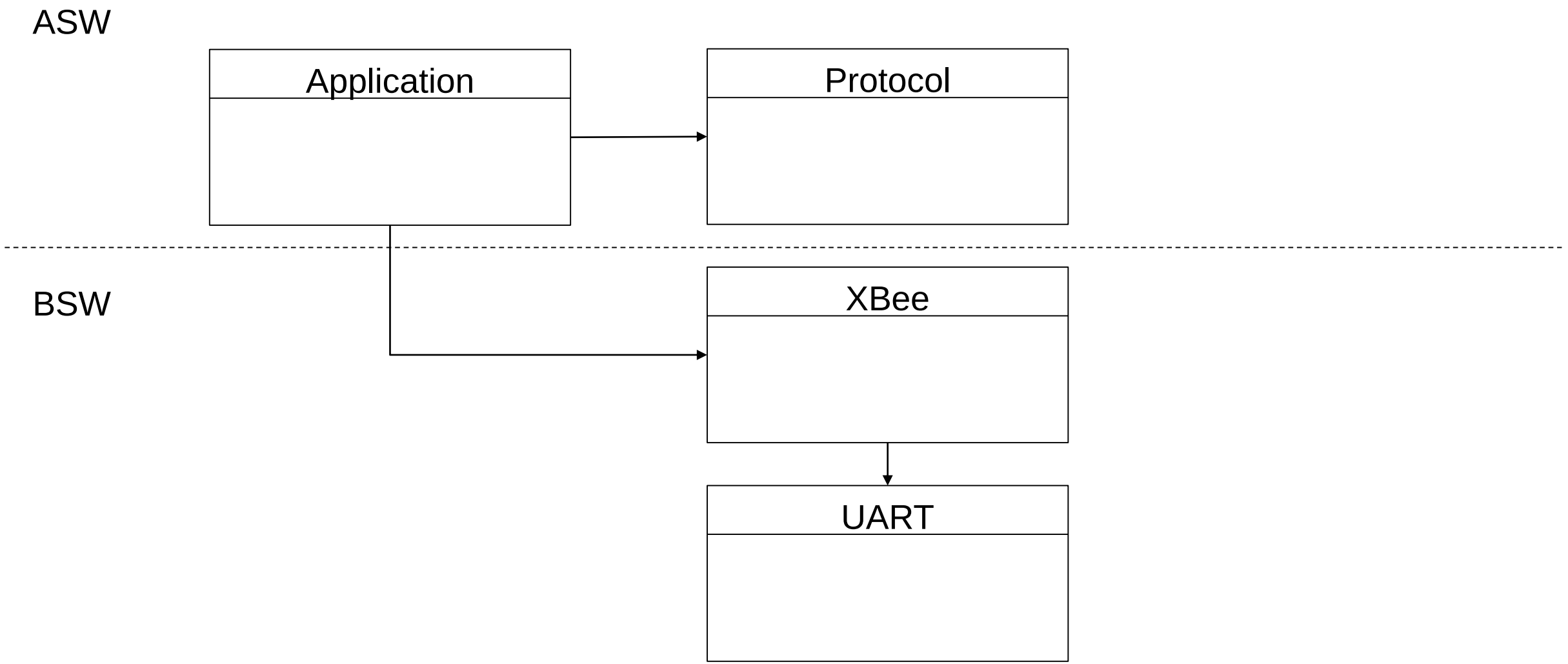
Transparent Mode:

- Data transmission

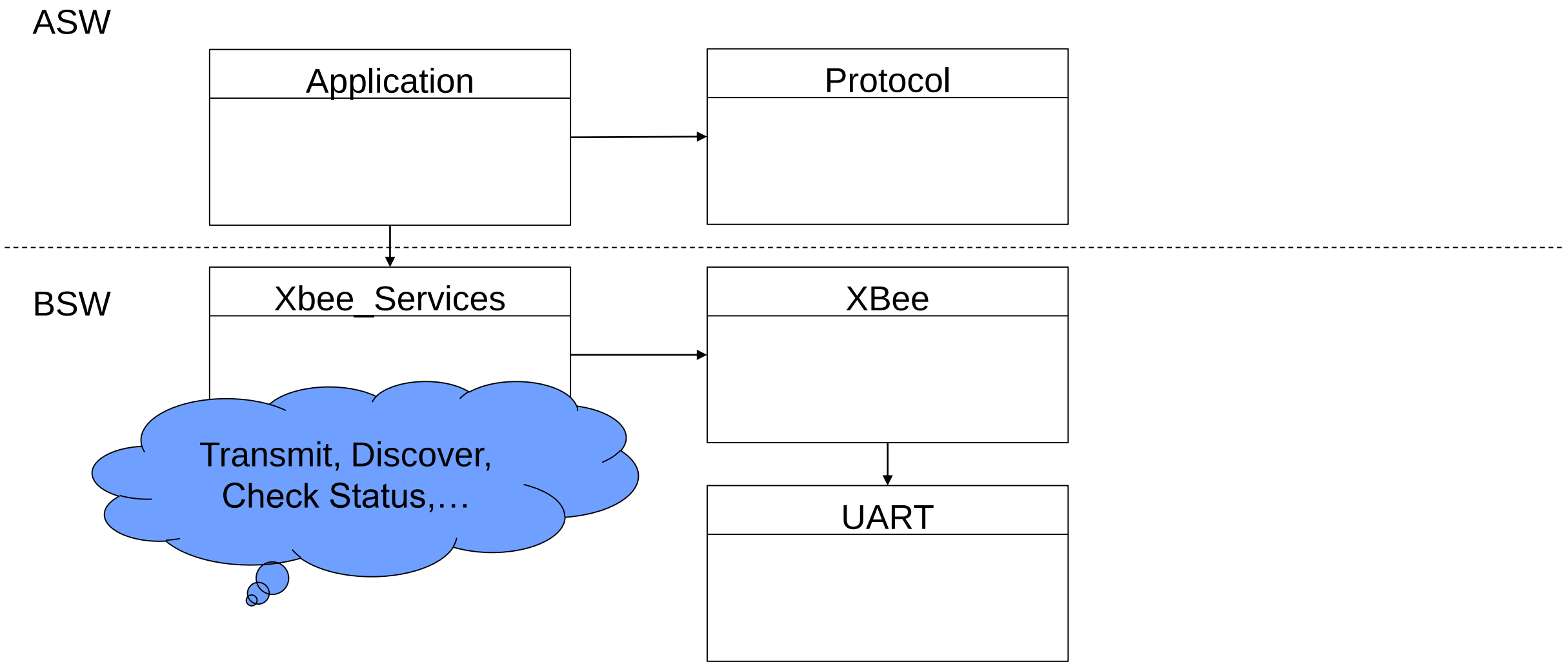
API Mode:

- Data Transmission
- Device Detection
- Device Configuration

Impact on the Driver : As-Is



Impact on the Driver : Extension of Service Layer

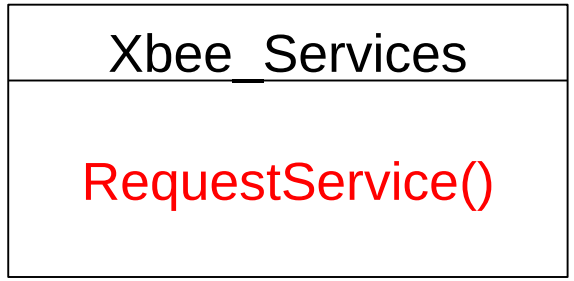


Concept for the Service Class

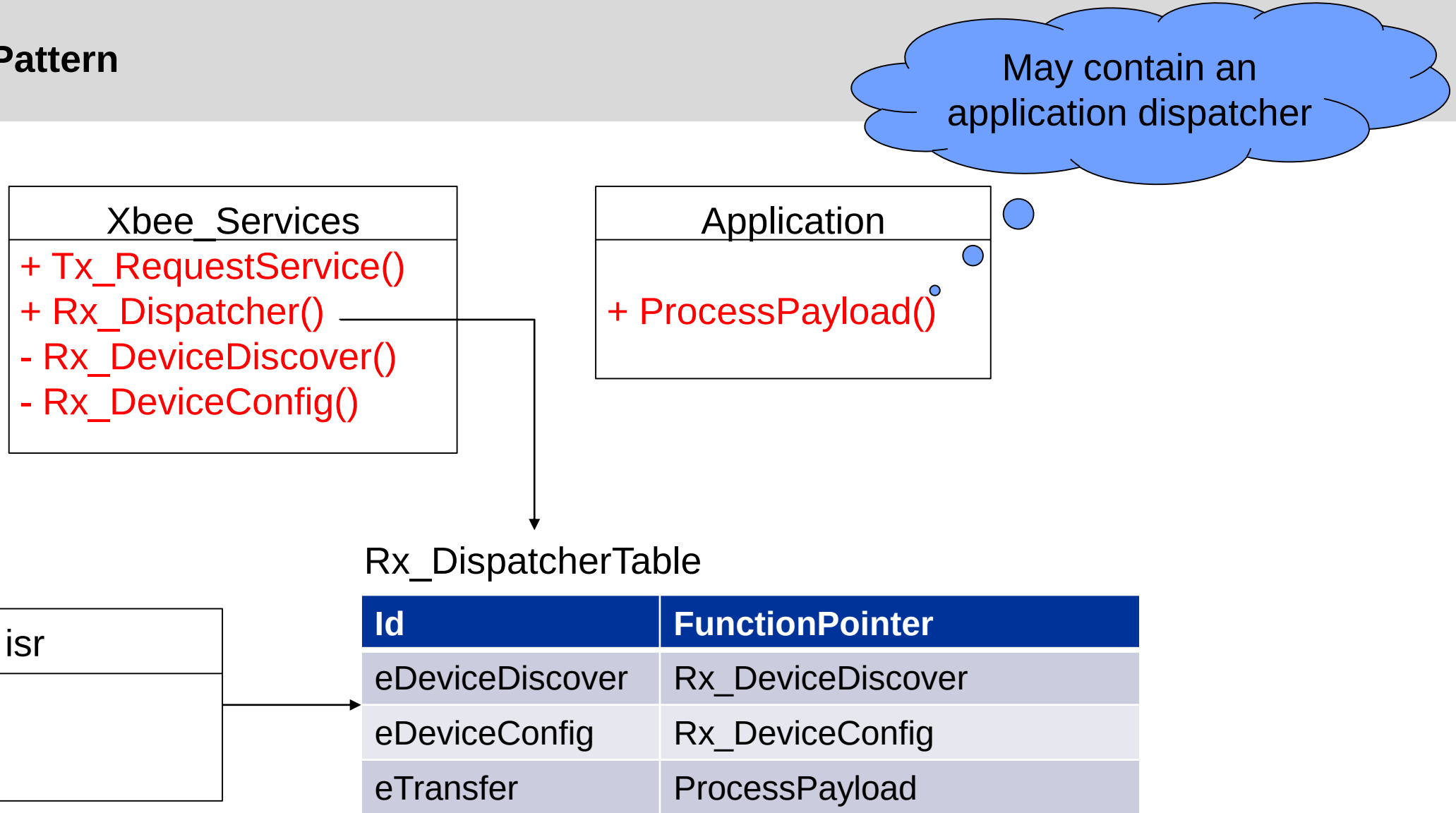
- We are moving from a simple Transmit / Receive API towards a Client / Server API

- RequestService consists of
  - Sending out data over the Xbee (simple wrapper)
  - Receiving and processing the answer (this is interesting)

- Some answers must be processed by the Xbee_Services class
  - Discover Devices
  - Get Device Status
  - …
- Some other answers must be passed on to the application
  - Transmission of User Data



Dispatcher Pattern

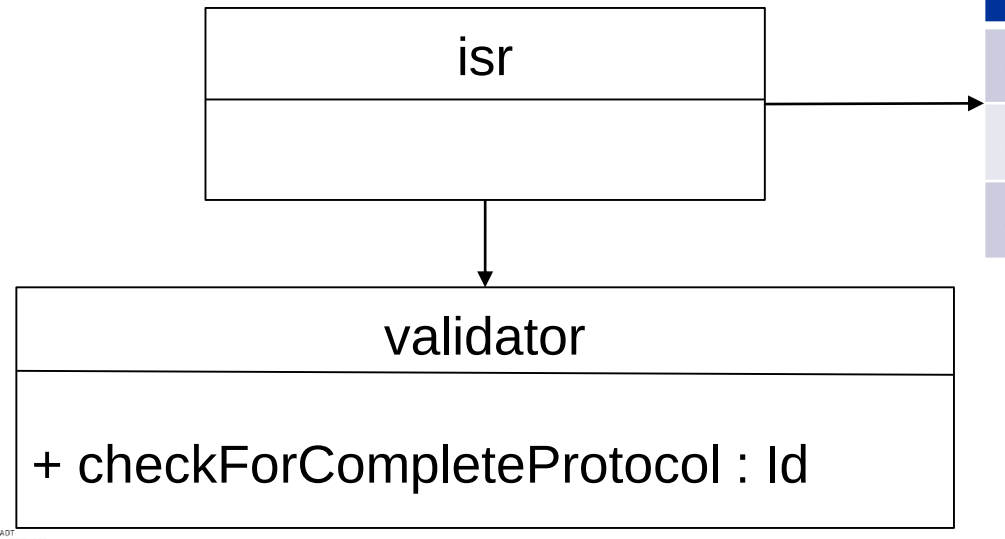


Id	FunctionPointer
eDeviceDiscover	Rx_DeviceDiscover
eDeviceConfig	Rx_DeviceConfig
eTransfer	ProcessPayload

Extension Dispatcher Pattern: Notifier / Validator / Dispatcher

Problem: When does the ISR know that the Rx_Dispatcher needs to be called?

- Option 1: Simple send every byte to the dispatcher and let the dispatcher check if a full protocol has been received. Bad cohesion!
- Option 2: Add another function pointer “validator()” to the service class, which checks if a complete protocol has been received. This validator is application specific and is called inside the isr. Furthermore this validator can extract the Id.

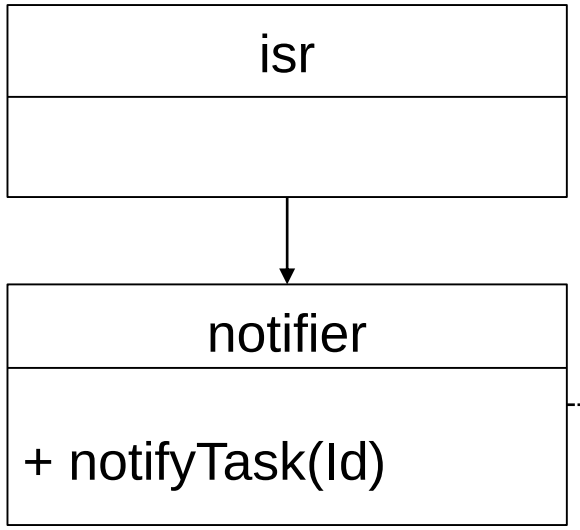


Id	FunctionPointer
eDeviceDiscover	Rx_DeviceDiscover
eDeviceConfig	Rx_DeviceConfig
eTransfer	ProcessPayload

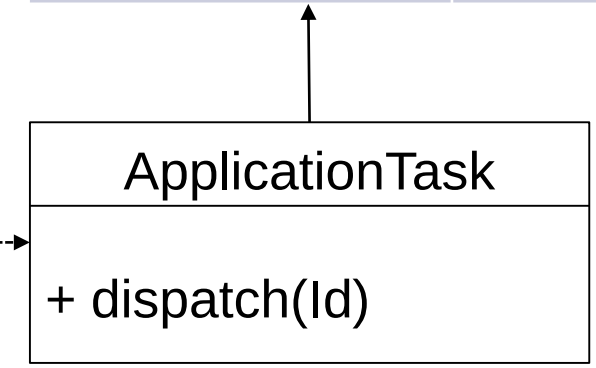
Extension Dispatcher Pattern: Notifier / Validator / Dispatcher

Next Problem: Now we would like to add a bit more flexibility and decoupling to the system. At the moment, the dispatcher is called in the interrupt context. Bad design, as the interrupt is blocked during this period.

- Option: Add a user defined notifier to the design, which will trigger a task by sending an event and thus change the context from isr to user task.



event



Id	FunctionPointer
eDeviceDiscover	Rx_DeviceDiscover
eDeviceConfig	Rx_DeviceConfig
eTransfer	ProcessPayload

Wrap-Up

- After getting all the drivers up and running, the application development was easy going

- Some problems could only be found in the integrated system
  - Engine driver limitations
  - Device communication sync
  - Blocking runnables

- Communication system still might need some extensions.

- Logging the output is as helpful as using the debugger
- Trust me – but don’t trust the debugger values

- Extending the communication to a 1-n link will be another project for the next semester!